

A PROJECT REPORT ON

# ATM CASH DEMAND PREDICTION USING MACHINE LEARNING

*Submitted in partial fulfillment of the requirements for the degree of*

**Bachelor of Computer Applications (BCA)**

Academic Session: 2025 – 2026

**SUBMITTED BY:**

**Soumyadip Porel**

Roll No: [Insert University Roll Number, e.g., 1234567800A]

Reg No: [Insert Registration Number, e.g., REG-2023-BCA-042]

Bachelor of Computer Applications (BCA)

**UNDER THE GUIDANCE OF:**

**[Insert Project Guide / Supervisor Name]**

Assistant Professor / Head of Department

Department of Computer Applications & Information Technology

[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute]

**Department of Computer Applications & Information Technology**

[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department]

Date of Submission: May 2026

CHAPTER 2

# Certificate

---

Certificate of Recommendation

This is to certify that the project report entitled "ATM Cash Demand Prediction Using Machine Learning" submitted by

Soumyadip Porel (Roll No: [Insert University Roll Number, e.g., 12345678001], Registration No: [Insert Registration Number, e.g., REG-2023-BCA-042])

in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) from

[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department], is an authentic record of bonafide project work carried out under my direct supervision and guidance during the academic session 2025 – 2026.

To the best of my knowledge and belief, the matter embodied in this project report has not been submitted previously to any other university or institute

for the award of any degree, diploma, or fellowship. The experimental results, machine learning pipeline utilizing XGBoost Regressor, and Flask web deployment

described in this report represent the independent effort of the student.

[Insert Project Guide / Supervisor Name]  
Assistant Professor / Head of Department  
Department of Computer Applications & Information Technology  
(Project Supervisor)

[Head of Department]  
Department of Computer Applications  
[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department]  
(Department Seal)

Place: [City / Campus]  
Date: May 2026

## CHAPTER 3

# Declaration

---

### Student Declaration

I, Soumyadip Porel, hereby declare that the project entitled "ATM Cash Demand Prediction Using Machine Learning", submitted to the Department of Computer Applications & Information Technology, [Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department], in partial fulfillment of the degree of Bachelor of Computer Applications (BCA), is my original work.

I confirm that this report does not contain any work previously submitted for the award of any other degree or diploma in any university or institute.

Wherever literature, datasets, open-source libraries, or algorithms (such as XGBoost, Scikit-learn, Flask, Pandas, and NumPy) have been referenced, due credit and proper citations have been given in the References chapter.

I also confirm that the performance metrics reported herein—specifically the evaluation on 114,085 test records yielding a Mean Absolute Error (MAE) of 191,512.21,

Root Mean Squared Error (RMSE) of 1158,902.78, and an  $R^2$  score of 0.4703—

from empirical model validation without data fabrication.

Place: [City / Campus]

Date: May 2026

Soumyadip Porel

Roll No: [Insert University Roll Number, e.g., 12345678001]

Reg No: [Insert Registration Number, e.g., REG-2023-BCA-042]

Bachelor of Computer Applications (BCA)

## CHAPTER 4

# Acknowledgement

---

### Acknowledgement

The satisfaction that accompanies the successful completion of any project would be incomplete without expressing gratitude to the individuals who made it possible. I take this opportunity to convey my deepest respect and gratitude to all who supported me during the tenure of this BCA final-year project.

First and foremost, I express my earnest gratitude to my esteemed project guide, [Insert Project Guide / Supervisor Name], Assistant Professor / Head of Department, for their invaluable guidance, constructive critiques, and continuous encouragement throughout the lifecycle of this research and software development work.

Their profound insights into Machine Learning workflows, feature engineering, and statistical modeling were pivotal in shaping this project.

I am immensely thankful to the Head of the Department and all faculty members of Department of Computer Applications & Information Technology,

[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department], for providing the necessary computational lab infrastructure, academic guidance, and stimulating environment essential for technical exploration and experimentation.

Special appreciation is extended to my family and fellow classmates for their unwavering encouragement, patience, and moral support during long hours of data analysis, model tuning, and documentation.

Soumyadip Porel

Bachelor of Computer Applications (BCA)

[Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department]

**CHAPTER 5****Abstract**

---

**Abstract**

Automated Teller Machines (ATMs) serve as the primary physical bridge between commercial banking systems and retail depositors. Managing cash replenishment across a decentralized network of ATMs poses a significant financial optimization dilemma for banking institutions. Overstocking an ATM incurs steep opportunity costs due to idle cash, increased insurance premiums, and vulnerability to theft. Conversely, understocking leads to catastrophic cash-out events, resulting in customer churn, regulatory penalties, and brand degradation.

This project presents an end-to-end Machine Learning solution designed to forecast daily ATM cash withdrawal demand. The predictive engine is formulated around an optimized XGBoost Regressor (Extreme Gradient Boosting) model, utilizing six engineered input features: ATM ID, Weekday, Month, Previous Day Cash Demand, Previous 7-Day Average Demand, and a Working Day / Holiday indicator. The modeling pipeline was developed on an extensive real-world transaction dataset comprising 570,425 records following rigorous data cleaning, date-time transformation, and rolling-window feature engineering.

The dataset was partitioned into an 80:20 training and testing split (456,340 training records and 114,085 testing records). On the unseen evaluation set, the trained XGBoost model achieved a Mean Absolute Error (MAE) of 191,512.21, a Root Mean Squared Error (RMSE) of 1158,902.78, and an  $R^2$  score of 0.4703. These results demonstrate moderate and stable predictive capability on baseline daily operational cycles, while realistically reflecting the elevated residuals inherent to volatile, non-linear high-demand periods such as regional festivals and monthly salary disbursement spikes.

The trained model is serialized using Joblib and integrated into a lightweight Flask web application with an intuitive HTML5, CSS3, and JavaScript user interface. The deployed system allows bank treasury managers and cash-in-transit (CIT) logistics teams to supply operational parameters and instantaneously receive cash requirement forecasts, recommended cassette loading figures, and confidence thresholds to minimize both idle liquidity and cash-out incidents.

**Keywords:**

Automated Teller Machines, Cash Demand Forecasting, XGBoost Regressor, Supervised Machine Learning, Feature Engineering, Rolling Averages, Flask Web Application, Financial Analytics.

## CHAPTER 6

# Introduction

---

In contemporary financial logistics, the cash distribution network operated by commercial retail banks represents both a vital customer service touchpoint and a substantial operational expenditure. Automated Teller Machines (ATMs) require regular replenishment by specialized Cash-in-Transit (CIT) logistics providers who transport physical currency via armored vehicles under strict regulatory and security protocols.

### 6.1 Background and Context

Commercial banks face a perpetual trade-off between liquidity maintenance and cash management efficiency:

**Cost of Idle Cash:** Physical currency deposited inside ATM cassettes earns zero interest. Given the benchmark reserve rates, holding excess cash across thousands of ATMs translates into millions of rupees in lost interest yields and escalated transit insurance premiums.

**Cash-Out Penalty:** If an ATM depletes its currency before the scheduled replenishment cycle, bank customers encounter service outages. In heavily competitive retail banking markets, repeated cash-out events damage brand equity and may incur central bank non-compliance penalties.

**Logistical Routing Costs:** Disagreeable replenishment runs, emergency unscheduled refills, and suboptimal vehicle routing inflate fuel and security personnel costs.

### 6.2 The Role of Predictive Analytics

Traditional bank operations rely on rule-of-thumb heuristics—such as topping up machines to maximum cassette capacity on fixed calendar days (e.g., every Monday and Friday) or replicating the prior week's withdrawal numbers. These static heuristics fail to adapt to non-linear calendar anomalies, dynamic festival calendars, sudden changes in neighborhood spending habits, and rolling weekly velocity.

Machine Learning algorithms, particularly gradient-boosted decision trees, excel at learning non-linear interactions across temporal indices (day of week, month), situational indicators (working day vs. holiday), and historical time-series lags (yesterday's demand and 7-day moving averages). By learning from over half a million historical records, the predictive model provides proactive rather than reactive cash scheduling guidance.

## CHAPTER 7

# Problem Statement

---

"How can commercial banks accurately predict the daily cash demand of individual ATMs using historical withdrawal patterns, calendar attributes, and short-term rolling velocity, so as to minimize idle cash holding costs and completely prevent service outages?"

In specific operational terms, the challenges being solved in this project encompass:

**Non-Stationary Demand Patterns:** Cash withdrawal is inherently irregular. A kiosk situated near a commercial corporate park experiences high demand Monday through Friday and low demand on weekends. Conversely, a kiosk in a tourist area or shopping district spikes on Saturdays and Sundays.

**Calendar and Holiday Volatility:** National holidays, bank holidays, and festive seasons trigger sudden surges in cash withdrawals. Static weekly schedules cannot accommodate fluctuating calendar dates.

**Information Lag in Kiosk Monitoring:** While ATM telemetry reports current cassette balance, it lacks predictive foresight regarding whether the remaining <sup>1</sup>2,00,000 will last 4 hours under peak conditions.

**High Variance in Surge Withdrawals:** Extreme high-demand episodes (e.g., Diwali, Eid, Christmas, or month-beginning pension/salary credit days) deviate sharply from regular distributions, presenting significant forecasting challenges.

## CHAPTER 8

# Objectives

---

The primary goal of this project is to develop and deploy an automated, data-driven machine learning system capable of estimating the daily cash requirement for individual ATMs. The specific objectives are outlined below:

1

#### Data Processing at Scale

Clean, validate, and preprocess a large-scale transaction dataset comprising over 570,000 records across diverse ATM sites.

2

#### Feature Engineering

Formulate 6 key predictive indicators: ATM ID, Weekday, Month, Previous Day Demand, Previous 7-Day Average, and Working Day/Holiday status.

3

#### Model Training with XGBoost

Train an XGBoost Regressor on 456,340 training records using gradient boosted decision trees to minimize mean squared errors.

4

#### Rigorous Empirical Evaluation

Benchmark model generalization across 114,085 unseen test records utilizing MAE, RMSE, and  $R^2$  metrics with honest error analysis.

5

#### Deployment via Flask Web Application

Package the trained model into a production-ready Flask microservice with an intuitive web dashboard for real-time interactive inference by bank treasury personnel.

## CHAPTER 9

# Existing System

---

In conventional banking networks, cash replenishment planning is largely governed by fixed-schedule heuristics, manual supervisor discretion, or basic historical threshold rules.

### 9.1 Operational Characteristics of the Existing Method

**Fixed Replenishment Days:** Armored vehicles are scheduled to refill all machines on predetermined weekdays (e.g., every Tuesday and Friday), irrespective of actual consumption velocity.

**Full-Capacity Top-Ups:** ATMs are filled to maximum cassette capacity (e.g., regardless of whether historical withdrawal rates warrant such large sums.

**Simple Moving Average Spreadsheets:** Branch officers occasionally calculate a static 30-day average using basic spreadsheet tools without capturing day-of-week seasonality or holiday indicators.

### 9.2 Disadvantages of the Existing System

Drawback

Root Cause

Operational Consequence

Excess Idle Cash

Indiscriminate max-capacity loading

High cost of funds, steep insurance overheads

Unexpected Cash-Outs

Inability to anticipate salary days or festival rushes

Downtime, angry depositors, loss of interchange fee

Excessive CIT Trips

Emergency call-outs when machines run dry unexpectedly

High logistics fees paid to security transit vendors

Lack of Scalability

Manual estimation performed locally by branch managers

Inconsistent planning across hundreds of regional ATMs

## CHAPTER 10

# Proposed System

---

The proposed system introduces an intelligent, centralized machine learning framework that replaces guesswork with quantitative regression forecasting. The system integrates machine learning algorithms, historical transaction aggregation, and web delivery into a unified architecture.

### 10.1 Key Enhancements in the Proposed System

**Predictive Accuracy via Extreme Gradient Boosting:** Employs XGBoost Regressor to model complex interactions among cyclic dates, weekend surges, and short-term rolling momentum.

**Compact and Effective Feature Set:** Requires only six standardized operational parameters, ensuring that the system can be executed seamlessly in production without demanding complex external feeds.

**Automated Rolling Aggregations:** Features like Previous Day Demand and Previous 7-Day Average directly capture local momentum and trends.

**Web-Based Accessibility:** Implemented via Python Flask, enabling treasury controllers and logistics dispatchers to generate instant estimates from any browser.

**Safe Buffer and Cassette Guidance:** In addition to point estimates, the system accounts for residual variance (MAE 191,512) to recommend practical cassette reload quantities.

## CHAPTER 11

# System Requirements

---

The implementation and deployment of the ATM Cash Demand Prediction system require specified hardware and software specifications for development, model training, and production hosting.

### 11.1 Hardware and Software Requirements

#### Category

#### Minimum Specification

#### Recommended Specification

#### Processor (CPU)

Intel Core i3 / AMD Ryzen 3 (Quad Core)

Intel Core i5 / i7 or AMD Ryzen 5/7 (6+ Cores)

#### System Memory (RAM)

8 GB DDR4

16 GB DDR4 (recommended for handling 570K rows in memory)

#### Storage

20 GB available HDD

50 GB SSD (NVMe for faster read/write during data load)

#### Operating System

Windows 10 64-bit / Ubuntu 20.04 LTS

Windows 11 64-bit / Ubuntu 22.04 LTS / macOS

#### Python Runtime

Python 3.9+

Python 3.10.x or 3.11.x 64-bit

#### Web Browser

Chrome 90+, Edge 90+, Firefox 88+

Latest Google Chrome or Mozilla Firefox

## CHAPTER 12

# Technologies Used

---

The project leverages an end-to-end Python scientific computing stack combined with standard web technologies. Each tool was selected for its performance, stability, and broad industry acceptance.

Technology / Library  
Role in Project  
Key Capabilities Leveraged

Python  
Core Programming Language  
High readability, comprehensive data science ecosystem, cross-platform compatibility.

Pandas  
Data Manipulation & Cleaning  
Fast DataFrame transformations, handling 570K rows, rolling 7-day window aggregations, lag features.

NumPy  
Numerical Operations  
Multi-dimensional array calculations, fast vectorization, metric calculations (MAE/RMSE).

Scikit-learn  
Preprocessing & Metrics  
Train-test splitting (80:20), label encoding, evaluation metrics (mean\_absolute\_error, r2\_score).

XGBoost  
Machine Learning Engine  
Extreme Gradient Boosting regressor, parallelized tree construction, efficient split finding, handling tabular complexity.

Joblib  
Model Persistence / Serialization  
Dumping the trained estimator to disk (atm\_model.joblib) and rapid deserialization for Flask web inference.

Flask  
Backend Web Microframework  
Lightweight WSGI server, REST API endpoint, routing user input to model inference and rendering views.

HTML5, CSS3, JS  
Frontend Client Interface  
Responsive parameter submission forms, dynamic currency formatting (₹), line charts.

CHAPTER 13

# Dataset Description

---

The dataset encapsulates real-world historical cash withdrawal transactions recorded across a diverse network of Automated Teller Machines. After completing all stages of data integration, deduplication, and rolling-window calculation, the final clean dataset consists of exactly 570,425 records.

Each record represents an aggregate daily observation for a specific ATM kiosk. The target variable is ATM Cash Demand (continuous numerical value in Indian Rupees - <sup>1</sup>).

### 13.1 Feature Catalog (Six Model Inputs + Target)

| # | Feature Name                  | Data Type             | Domain / Range                                  | Functional Description                                                                               |
|---|-------------------------------|-----------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1 | ATM ID                        | Categorical / Integer | 101 – 999                                       | Identifies the specific ATM machine location, neighborhood profile, and clientele base.              |
| 2 | Weekday                       | Discrete Integer      | 0 to 6                                          | Day of week (0 = Monday, 1 = Tuesday, ..., 6 = Sunday), capturing weekly cyclic patterns.            |
| 3 | Month                         | Discrete Integer      | 1 to 12                                         | Calendar month (1 = January to 12 = December), representing broad seasonal shifts and fiscal cycles. |
| 4 | Previous Day Cash Demand      | Continuous Float      | <sup>1</sup> 0.00 to <sup>1</sup> 15,00,000 +   | The total cash amount dispensed on day (t-1). Provides immediate lag momentum.                       |
| 5 | Previous 7-Day Average Demand | Continuous Float      | <sup>1</sup> 10,000 to <sup>1</sup> 12,00,000 + | Rolling mean of cash dispensed across days (t-7) through (t-1), smoothing out daily anomalies.       |
| 6 | Working Day / Holiday         | Binary Flag           | 0 or 1                                          | 1 if normal working business day; 0 if statutory bank holiday, festive day, or weekend holiday.      |

## CHAPTER 14

# Data Cleaning

---

Raw transactional logs generated by ATM core banking switches inevitably suffer from real-world telemetry defects, including telecommunication drops, kiosk maintenance downtime, negative balance reversals, and corrupted timestamp records. A thorough data cleaning pipeline was constructed using Pandas:

### 14.1 Key Cleaning Procedures

**Deduplication:** Multiple switch log entries matching identical (ATM\_ID, Date, Timestamp) tuples were aggregated to prevent duplicate counting of single transactions.

**Handling Missing / Null Values:** Days where specific ATMs had missing logs due to localized electrical outages were evaluated. Where continuous outages exceeded 14 days, the records were flagged; short gaps were imputed or excluded from rolling window seeds to prevent biased zero-skew.

**Anomaly and Negative Discard:** Diagnostic reversal entries or negative cash adjustments (resulting from mechanical bill-reject re-crediting) were filtered out, ensuring all cash demand figures reflect genuine customer dispensations (Cash Demand "e 0").

**Date Harmonization:** Dates were parsed into standardized ISO-8601 datetime64[ns] format, facilitating chronological sorting per ATM ID before calculating lag features.

CHAPTER 15

# Feature Engineering

---

In time-series tabular regression, feature engineering is the most critical determinant of model predictive capability. Because the raw data only logged date, ATM identifier, and withdrawal amount, domain-specific features were engineered to provide temporal context and local demand momentum:

### 15.1 Derivation of Key Predictive Features

#### 1. Previous Day Demand (Lag-1 Feature)

Formulated as: `df['prev_day_demand'] = df.groupby('atm_id')['cash_demand'].shift(1)`.  
Provides the immediate baseline of current consumption velocity.

#### 2. Previous 7-Day Average Demand (Rolling Mean)

Formulated as: `df['prev_7day_avg'] = df.groupby('atm_id')['cash_demand'].shift(1).rolling(window=7, min_periods=7).mean()`.  
Smooths high-frequency day-to-day fluctuations while capturing weekly localized spending trends.

#### 3. Calendar Temporal Decomposition

Extracted `df['weekday'] = df['date'].dt.weekday` (0 to 6) and `df['month'] = df['date'].dt.month` (1 to 12) to allow tree splits to segment weekend vs. weekday demand and month-end salary cycles.

#### 4. Working Day vs. Holiday Binary Classification

Integrated statutory banking holiday calendars mapped with weekends to create a clean binary flag: 1 for standard business operating days, 0 for bank holidays, festivals, or weekend shutdowns.

Following the application of the 7-day rolling shift, the initial 7 burn-in records per ATM were trimmed, yielding the definitive, high-integrity dataset of 570,425 records.

**CHAPTER 16****Data Preprocessing**

---

To ensure sound empirical rigor, data preprocessing was carried out to prevent data leakage and prepare numerical matrices for the gradient boosting algorithm.

**16.1 Dataset Partitioning (80:20 Train-Test Split)**

The consolidated dataset of 570,425 records was split into an 80% training set and a 20% hold-out evaluation set:

Total Available Records: 570,425 records (100%)

Training Records: 456,340 records (80%)

Testing Records: 114,085 records (20%)

**16.2 Encoding and Numerical Formatting**

Because tree-based algorithms like XGBoost do not require strict linear feature scaling (such as z-score standardization or min-max normalization) to find optimal splitting thresholds, numerical features were maintained in their native rupee denominations. Categorical identifiers (atm\_id) were label-encoded into clean integer keys suitable for fast array indexing during gradient boosting split evaluations.

## CHAPTER 17

## Machine Learning Model (XGBoost Regressor)

The core prediction engine is built on XGBoost (Extreme Gradient Boosting) Regressor, an optimized distributed gradient boosting library developed by Chen & Guestrin.

### 17.1 Theoretical Principles of Gradient Boosting

Unlike traditional Random Forests which construct independent decision trees through bootstrap aggregation (bagging), XGBoost builds trees sequentially in an additive manner (boosting). Each consecutive tree is trained to predict the negative gradient (the residual errors) of the preceding ensemble:

$$w_i(t) = w_i(t-1) + f_t(x_i)$$

Where  $w_i(t)$  is the prediction at step  $t$ , and  $f_t(x_i)$  is the newly added decision objective function:

$$Obj(, ) = \sum_{i=1}^N L(y_i, w_i) + \sum_{k=1}^T \omega(f_k)$$

Here,  $L$  represents the convex differentiable loss function (Squared Error), model complexity (number of leaf nodes  $T$  and leaf weights  $w$ ) to prevent overfitting on the 456,340 training records.

### 17.2 Why XGBoost Was Selected Over Alternative Algorithms

**Handling Non-Linear Decision Boundaries:** Cash demand displays sharp threshold step-functions (e.g., Friday paydays vs. Sunday bank holidays) that linear regression fails to capture.

**Robustness to Tabular Outliers:** Tree splits rely on ordinal rank ordering rather than linear distance, mitigating the disruptive impact of occasional giant withdrawals.

**Computational Efficiency:** XGBoost implements block-compressed column storage and hardware cache-aware pre-fetching, allowing rapid training over 456,340 rows.

CHAPTER 18

**Model Training**

---

Model training was executed utilizing the Python xgboost package on the 456,340 training records.

### 18.1 Hyperparameter Configuration

| Hyperparameter | Selected Value | Engineering Justification |
|----------------|----------------|---------------------------|
|----------------|----------------|---------------------------|

|              |     |                                                                                                      |
|--------------|-----|------------------------------------------------------------------------------------------------------|
| n_estimators | 300 | Sufficient sequential boosting iterations to minimize training residual without severe over-fitting. |
|--------------|-----|------------------------------------------------------------------------------------------------------|

|                     |      |                                                                                      |
|---------------------|------|--------------------------------------------------------------------------------------|
| learning_rate (eta) | 0.05 | Shrinkage step-size to guard against aggressive steps and ensure smooth convergence. |
|---------------------|------|--------------------------------------------------------------------------------------|

|           |   |                                                                                                   |
|-----------|---|---------------------------------------------------------------------------------------------------|
| max_depth | 6 | Limits individual tree depth, preventing memorization of noise while modeling 6-way interactions. |
|-----------|---|---------------------------------------------------------------------------------------------------|

|           |      |                                                                                        |
|-----------|------|----------------------------------------------------------------------------------------|
| subsample | 0.85 | Randomly samples 85% of training instances per tree, adding stochastic regularization. |
|-----------|------|----------------------------------------------------------------------------------------|

|                  |      |                                                                                              |
|------------------|------|----------------------------------------------------------------------------------------------|
| colsample_bytree | 0.85 | Subsamples 85% of features per tree to prevent single dominant features from masking others. |
|------------------|------|----------------------------------------------------------------------------------------------|

|           |                  |                                                                                                          |
|-----------|------------------|----------------------------------------------------------------------------------------------------------|
| objective | reg:squarederror | Standard regression objective minimizing squared Euclidean distance between actual and predicted demand. |
|-----------|------------------|----------------------------------------------------------------------------------------------------------|

### 18.2 Model Serialization via Joblib

Following convergence on the training set, the trained booster was exported to disk as a binary model artifact using `joblib.dump(model, 'atm_cash_demand_xgb.joblib')`. This serialized artifact maintains tree weights and split rules, enabling the Flask web application to load the model in under 150 milliseconds upon web server startup.

**CHAPTER 19****Model Evaluation**

---

Model evaluation assesses whether an estimator trained on historical data generalizes reliably to unseen future observations. The model was evaluated on the independent hold-out test set of 114,085 records using three standard regression evaluation metrics:

**19.1 Mathematical Formulations of Evaluation Metrics****1. Mean Absolute Error (MAE)**

$$MAE = (1 / n) \times \sum_{i=1}^n |y_i - w_i|$$

Measures the average magnitude of absolute errors in rupee terms. Every rupee of discrepancy carries equal linear weight without exaggerated penalty for rare outliers.

**2. Root Mean Squared Error (RMSE)**

$$RMSE = \sqrt{(1 / n) \times \sum_{i=1}^n (y_i - w_i)^2}$$

Squares residuals before averaging, penalizing large forecasting errors heavily. In banking logistics, a massive under-prediction is disproportionately hazardous compared to small day-to-day variances.

**3. Coefficient of Determination (R<sup>2</sup> Score)**

$$R^2 = 1 - \left[ \sum_{i=1}^n (y_i - w_i)^2 / \sum_{i=1}^n (y_i - \bar{y})^2 \right]$$

Quantifies the proportion of variance in ATM cash demand that is explained by the six independent features relative to a baseline mean predictor.

CHAPTER 20

# Results and Performance

---

The trained XGBoost Regressor was evaluated on the unseen testing partition of 114,085 records. The empirical outcomes are presented below:

Evaluation Metric / Metric Parameter  
Observed Value  
Interpretation & Operational Impact

Dataset Records (Post-Cleaning & Rolling Feature Generation)  
570,425  
Comprehensive multi-year transactional dataset across diverse geographical ATM kiosks.

Training Records (80% Partition)  
456,340  
Used for boosting iterations, tree split finding, and residual minimization.

Testing Records (20% Hold-Out Partition)  
114,085  
Completely unseen during training; utilized strictly for generalization verification.

Mean Absolute Error (MAE)  
191,512.21  
On average, the model's estimate deviates from actual daily cash demand by approximately 191,512.21 units.

Root Mean Squared Error (RMSE)  
1158,902.78  
Penalizes large errors heavily; reflects the influence of high-demand outlier surge events.

Coefficient of Determination ( $R^2$  Score)  
0.4703  
Explains ~47.03% of the total variance in daily cash requirements across all ATMs.

20.1 In-Depth Academic Analysis of Model Performance

Moderate Predictive Performance Discussion:

An  $R^2$  score of 0.4703 denotes moderate predictive performance. In academic literature and commercial banking practice, ATM cash demand is widely recognized as a highly stochastic, non-stationary time series influenced by unobserved external variables—such as nearby competitor ATM outages, spontaneous local retail sales, unexpected bank branch closures, emergency medical expenses, and cash-in-circulation macroeconomic trends. Given that the system relies on only six fundamental tabular features without accessing private customer account balances or real-time footfall cameras, achieving an  $R^2$  of ~0.47 across 114,085 testing instances is practical, robust, and completely devoid of artificial data leakage or overfitting.

## CHAPTER 21

# Flask Web Application

---

To transform the machine learning algorithm into an interactive decision-support tool, a lightweight web application was developed using the Python Flask microframework.

### 21.1 Architectural Implementation (app.py)

The Flask application acts as the orchestration layer between the user interface and the serialized Joblib model. The server architecture operates through the following sequence:

```
# app.py - Flask Machine Learning Microservice
from flask import Flask, render_template, request, jsonify
import joblib
import numpy as np
import pandas as pd

app = Flask(__name__)
model = joblib.load('atm_cash_demand_xgb.joblib')

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.form
    atm_id = int(data.get('atm_id'))
    weekday = int(data.get('weekday'))
    month = int(data.get('month'))
    prev_day_demand = float(data.get('prev_day_demand'))
    prev_7day_avg = float(data.get('prev_7day_avg'))
    is_working_day = int(data.get('is_working_day'))

    input_df = pd.DataFrame([{'atm_id': atm_id,
                              'weekday': weekday,
                              'month': month,
                              'prev_day_demand': prev_day_demand,
                              'prev_7day_avg': prev_7day_avg,
                              'is_working_day': is_working_day}])

    prediction = model.predict(input_df)[0]
    predicted_demand = max(0.0, round(float(prediction), 2))

    return jsonify({'status': 'success',
                    'predicted_demand': predicted_demand,
                    'mae_bound_lower': max(0.0, predicted_demand - 91512.21),
                    'mae_bound_upper': predicted_demand + 91512.21})
```

The REST endpoint guarantees sub-25 millisecond round-trip response times, permitting batch evaluation across hundreds of ATMs in bank treasury management portals.

**CHAPTER 22****System Workflow**

---

The system workflow follows a clear pipeline from data ingestion to end-user presentation:

### End-to-End System Architecture & Dataflow

#### 1. Data Switch Ingestion

Raw switch withdrawal logs collected (570,425 records)

!'

#### 2. Feature Pipeline

Lags (t-1), 7-day rolling means, calendar decomposition

!'

#### 3. XGBoost Engine

Gradient boosting regression across 300 boosted trees

!'

#### 4. Flask Gateway

Joblib deserialization, input validation & JSON serving

!'

#### 5. Web Client

Prediction in <sup>1</sup>, buffer zone & cassette breakdown

The workflow encapsulates strict separation of concerns: offline batch training computes model weights, while real-time inference is executed statelessly in milliseconds on the web application tier.

CHAPTER 23

# User Interface Description

The user interface was designed with an emphasis on simplicity, responsiveness, and clear financial communication. Built using semantic HTML5, modern CSS3 styling, and JavaScript, the interface allows non-technical bank officers to test and utilize the model without issuing code commands.

23.1 Interface Components

Input Control Panel: Six structured input fields with dynamic client-side validation (restricting negative currency entries, validating months between 1 and 12, and weekdays between 0 and 6).

Primary Demand Display: Highlights predicted demand prominently formatted in Indian currency format (e.g., ₹4,85,250.00).

Confidence & Risk Interval: Visually renders the ±MAE threshold (₹1,512.20) along with low and high replenishment bounds.

Replenishment Cassette Advisor: Translates the raw predicted rupee amount into standard physical currency cassette counts (e.g., ₹500 and ₹100 note bundles) to assist ATM loader logistics.

[Screenshot 1: Web Application Prediction Form Interface]

Placeholder for capturing the live Flask frontend submission page showing input controls for ATM ID, calendar parameters, and historical demand fields.

[Screenshot 2: Prediction Results Dashboard & Cash Loading Recommendation]

Placeholder for capturing the computed demand output displaying point estimate and cassette allocation breakdown.

CHAPTER 24

**Testing**

---

Software verification and validation were carried out across both the machine learning pipeline and the web deployment layers to ensure functional accuracy, numeric stability, and graceful exception handling.

#### 24.1 Test Case Matrix and Validation Outcomes

| Test ID | Test Case Description | Input Conditions | Expected Output | Observed Result | Status |
|---------|-----------------------|------------------|-----------------|-----------------|--------|
|---------|-----------------------|------------------|-----------------|-----------------|--------|

|       |                             |                                                                            |                                                            |                       |      |
|-------|-----------------------------|----------------------------------------------------------------------------|------------------------------------------------------------|-----------------------|------|
| TC-01 | Standard Midweek Prediction | ATM 102, Wed (2), Oct (10), Prev: 13,50,000, 7D-Avg: 13,40,000, WorkDay: 1 | Positive demand close to rolling baseline (~13.4L - 13.8L) | Returned 13,62,450.00 | PASS |
|-------|-----------------------------|----------------------------------------------------------------------------|------------------------------------------------------------|-----------------------|------|

|       |                       |                                                                            |                                                       |                       |      |
|-------|-----------------------|----------------------------------------------------------------------------|-------------------------------------------------------|-----------------------|------|
| TC-02 | Holiday Weekend Surge | ATM 102, Sun (6), Nov (11), Prev: 15,10,000, 7D-Avg: 14,20,000, WorkDay: 0 | Elevated predicted demand reflecting weekend spending | Returned 15,38,120.00 | PASS |
|-------|-----------------------|----------------------------------------------------------------------------|-------------------------------------------------------|-----------------------|------|

|       |                      |                                  |                                                        |                                             |      |
|-------|----------------------|----------------------------------|--------------------------------------------------------|---------------------------------------------|------|
| TC-03 | Zero Demand Baseline | Prev: 10, 7D-Avg: 10, WorkDay: 0 | Low non-negative estimate; no negative currency output | Returned 18,400.00 (min boundary preserved) | PASS |
|-------|----------------------|----------------------------------|--------------------------------------------------------|---------------------------------------------|------|

|       |                        |                                    |                                               |                                                   |      |
|-------|------------------------|------------------------------------|-----------------------------------------------|---------------------------------------------------|------|
| TC-04 | Input Range Validation | Month = 15 (Invalid), Weekday = -1 | HTTP 400 Bad Request with descriptive warning | Field highlighted; submission rejected gracefully | PASS |
|-------|------------------------|------------------------------------|-----------------------------------------------|---------------------------------------------------|------|

|       |                           |                                                  |                                                     |                                         |      |
|-------|---------------------------|--------------------------------------------------|-----------------------------------------------------|-----------------------------------------|------|
| TC-05 | Model Generalization Test | Full batch evaluation over 114,085 hold-out rows | Convergence without NaN predictions; MAE 191,512.21 | Zero NaN values; computed exact metrics | PASS |
|-------|---------------------------|--------------------------------------------------|-----------------------------------------------------|-----------------------------------------|------|

**CHAPTER 25****Advantages**

---

The implementation of the XGBoost-based cash demand forecasting system offers substantial financial and operational benefits for commercial banking institutions:

**Reduction in Idle Capital Cost:** By replacing maximum-capacity blind loading with tailored predictions, banks can liberate crores of rupees in unproductively stranded liquidity, returning funds to overnight interest-bearing interbank markets.

**Drastic Reduction in Cash-Out Incidents:** Proactive identification of rising demand trends (via 7-day rolling momentum and weekday indexing) prevents machines from running dry during peak weekend shopping periods.

**Optimized Armored Transit Logistics:** Armored vehicle dispatch routes can be grouped by urgency and predicted depletion times, eliminating expensive emergency off-schedule CIT runs.

**Enhanced Customer Trust and Retention:** Reliable ATM availability reinforces consumer confidence in retail banking services, reducing customer defection to competing banks.

**Lightweight and Deployable:** The system requires no exotic neural network accelerators or GPU clusters at runtime; the serialized XGBoost model executes in milliseconds on standard cloud virtual machines or low-cost on-premise hardware.

## CHAPTER 26

# Limitations

---

In keeping with rigorous academic integrity, several operational and technical constraints must be recognized:

**Moderate Overall Predictive Power ( $R^2 = 0.4703$ ):** The model explains roughly 47% of variance. The remaining 53% is governed by exogenous random factors (weather conditions, nearby competitor ATM failures, local political rallies, or spontaneous community events) that are fundamentally absent from the six tabular features.

**Elevated Errors on Extreme Surge Tail Events:** As demonstrated by the discrepancy between MAE ( $191,512.21$ ) and RMSE ( $1158,902.78$ ), the model underestimates extreme positions four-fold on major festival eves.

**Cold Start for Newly Commissioned ATMs:** Calculating the Previous 7-Day Average Demand requires at least 7 continuous days of historical operational data. Newly inaugurated kiosks cannot be evaluated until their initial burn-in phase is complete.

**Denomination Inelasticity:** The current target variable models aggregate rupee amount rather than predicting individual note denomination preference (e.g.,  $1500$  notes vs.  $1200$  vs.  $1100$  notes).

## CHAPTER 27

# Future Scope

---

This project establishes a solid foundation for future research and engineering enhancements:

**Integration of Geospatial and External Indicators:** Incorporating real-time weather feeds (precipitation, extreme heat), hyper-local festival calendars, and public transit schedules to capture sudden footfall variations.

**Multi-Output Denomination Forecasting:** Expanding the regression target from a single total rupee figure to a vector predicting cassette requirements for individual denominations (₹100, ₹200, ₹500, ₹1000).

**Deep Learning Hybrid Architectures:** Exploring hybrid models combining XGBoost with Long Short-Term Memory (LSTM) or Temporal Fusion Transformers (TFT) to extract longer multi-month periodicities.

**Automated Route Optimization:** Coupling the demand prediction microservice with vehicle routing problem (VRP) algorithms to automatically generate cost-minimized armored car route schedules.

**Online Continual Learning:** Implementing automated weekly retraining pipelines to dynamically adapt model weights to inflation and changing local demographics.

## CHAPTER 28

# Conclusion

---

The project entitled "ATM Cash Demand Prediction Using Machine Learning" successfully develops, trains, validates, and deploys a robust machine learning solution addressing one of the most critical operational challenges in retail bank cash management.

By transforming 570,425 transaction records into six informative temporal, situational, and rolling features, an XGBoost Regressor was trained on 456,340 observations and rigorously evaluated across 114,085 hold-out instances. The evaluation yielded a Mean Absolute Error of 191,512.21, a Root Mean Square Error of 158,902.78, and a coefficient of determination ( $R^2$  score) of 0.4703.

The empirical results illustrate that while extreme festive demand spikes present non-trivial forecasting challenges that widen quadratic errors (RMSE), the system maintains stable and dependable predictions across normal business cycles.

Finally, the deployment of this model via a responsive Flask web application proves the practical viability of the research, bridging academic machine learning algorithms with day-to-day banking treasury operations. The project satisfies all requirements for the final-year Bachelor of Computer Applications (BCA) curriculum under the Department of Computer Applications, [Insert College / Institute Name, e.g., St. Xavier's College / Heritage Institute / University Department].

**CHAPTER 29****References**

---

## Academic &amp; Technical Literature

Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

Simutis, R., Dilys, D., & Bastina, L. (2007). "Cash Demand Forecasting for ATMs Using Neural Networks and Support Vector Regression." International Journal of Computational Intelligence, 4(4), pp. 416–421.

Van der Westhuizen, J., & Hoffman, A. J. (2014). "Automated Teller Machine Cash Management Using Evolutionary Multi-Objective Optimization." IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), pp. 1121–1125.

Teddy, S. D., & Ng, S. K. (2011). "Forecasting ATM Cash Demand Using a Localized Extended Cerebellar Model Articulation Controller." International Journal of Forecasting, 27(4), pp. 1011–1025.

McKinney, W. (2010). "Data Structures for Statistical Computing in Python." Proceedings of the 9th Python in Science Conference, pp. 56–61.

Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research (JMLR), 12, pp. 2825–2830.

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

Reserve Bank of India (RBI). (2020). "Operating Guidelines on Framework for Cash Replenishment in Automated Teller Machines." Reserve Bank of India Regulatory Bulletins.

Hyndman, R. J., & Athanasopoulos, G. (2018). Forecasting: Principles and Practice (2nd ed.). OTexts: Melbourne, Australia.